

Interview Questions — Java - Core Java

Total Questions: 72

1. What is the main difference between the JDK, JRE, and JVM in Java?

Hint: Think about which one is for development, which one is for running programs, and which one is the actual engine.

2. Why is Java considered platform-independent?

Hint: Focus on the role of the JVM in running Java bytecode.

3. Describe the steps to install the JDK and set up a Java development environment using Eclipse or IntelliJ.

4. Write a simple Java program that prints 'Hello Digital Wallet!' to the console.

5. Explain the structure of a basic Java class. What is the purpose of the main method?

6. What are primitive data types in Java? Give three examples.

Hint: Think about int, double, boolean, etc.

7. How are primitive types different from non-primitive types in Java?

Hint: Consider memory usage and built-in vs user-defined types.

8. Write a Java statement to declare a variable to store a user's wallet balance as a decimal number.

9. What is type casting in Java? Give an example of both implicit and explicit casting.

10. What is the difference between the == and != operators in Java?

11. How would you use an if-else statement in Java to check if a user's wallet balance is less than 100 and print a warning message?

Hint: Remember the syntax: if (condition) { ... } else { ... }

12. What is the purpose of the switch statement in Java, and when would you use it instead of multiple if-else statements?

13. Rewrite this logic using a ternary operator: If balance 100, print 'Low Balance', else print 'Sufficient Balance'.

Hint: The ternary operator uses the format: condition ? value1 : value2

14. How can you relate the use of conditional statements in Java to SQL WHERE clauses?

15. What is the difference between a for loop and a while loop in Java?

Hint: Think about when each loop checks its condition.

16. Write a Java loop that prints the numbers 1 to 5 using a for loop.

17. How would you use a loop to display a summary of monthly transactions stored in an array?

18. What is the purpose of the break and continue statements in loops?

19. How do you declare and initialize an array to store the last 10 transaction amounts in Java?

Hint: Consider the syntax for creating an array of a specific size.

20. Explain the difference between single-dimensional and multi-dimensional arrays in Java.

21. Write code to traverse an array of transaction amounts and print each one.

22. How do arrays in Java help model database tables (rows)?

23. What is Object-Oriented Programming (OOP), and why is it preferred over procedural programming?

Hint: Think about code organization and reusability.

24. Give a real-world analogy for objects and classes, using a User and Wallet as examples.

25. How do you define a class in Java, and how do you create an object of that class?

26. What is the difference between instance variables and static variables in a class?

27. Write a method in a Wallet class called addMoney that takes an amount as a parameter and updates the balance.

28. How are classes in Java similar to tables in a database, and how are methods similar to CRUD operations?

29. What is a constructor in Java? How does it differ from a regular method?

Hint: Think about how and when each is called.

30. Write two constructors for a User class: one with no parameters and one that takes a name and balance.

31. Explain method overloading with an example.

32. How is method overloading similar to having different forms of INSERT queries in a database?

33. What is inheritance in Java? Give an example using User and PremiumUser classes.

34. What is the purpose of the super and this keywords in Java inheritance?

Hint: One refers to the parent class, the other to the current instance.

35. Explain the difference between single, multilevel, and hierarchical inheritance in Java.

36. What is polymorphism in Java? Distinguish between compile-time and runtime polymorphism.

37. How does method overriding work in Java? Provide a simple example.

38. Describe a scenario where you would use dynamic method dispatch.

39. What is encapsulation, and how do access modifiers help achieve it in Java?

Hint: Think about data hiding and controlling access.

40. How do you use getters and setters to access private variables in a class?

41. What is the difference between an abstract class and an interface in Java?

42. Write an abstract class called Payment with an abstract method processPayment.

43. How does abstraction in Java help define API contracts between different layers of an application?

44. What is the difference between String, StringBuilder, and StringBuffer in Java?

Hint: Consider mutability and thread safety.

-
45. Write code to mask the first four characters of a wallet ID (e.g., 'WAL1234' → '***1234').
-
46. How would you compare two strings for equality in Java?
-
47. Why is exception handling important in Java applications?
-
48. Write a try-catch block that catches an exception if a user tries to withdraw more than their wallet balance.
-
49. What is the purpose of the finally block in exception handling?
-
50. Explain the difference between throw and throws in Java.
-
51. How do you create a custom exception class called `InsufficientFundsException`?
-
52. Write code to handle multiple possible exceptions in a transaction process using multiple catch blocks.
-
53. What classes would you use to read from and write to a text file in Java?
-
54. Write code to save a list of transaction logs to a file using `FileWriter`.
-
55. How does file I/O in Java simulate basic data persistence before using a database?
-
56. What is the Java Collections Framework, and why is it useful?
- Hint:** Think about storing and managing groups of objects.
-
57. How would you use an `ArrayList` to store a list of transaction objects in Java?
-
58. What is the difference between a `List` and a `Set` in Java Collections?
-
59. How does a `HashMap` work, and when would you use it to store transaction details?
-
60. Write code to iterate over a `HashMap` of transaction IDs and print each transaction's details.
-
61. How would you sort a collection of transaction amounts in Java?
-
62. Explain how keys and values in a `Map` relate to columns and records in a database.
-
63. What is a thread in Java, and what are the main steps in its lifecycle?
-

-
64. How would you use a thread to perform a background task, such as updating cashback every 10 seconds?
-
65. What is the Runnable interface, and how is it used to create threads in Java?
-
66. How does parallelism using threads improve backend operations in applications like a digital wallet?
-
67. Describe how you would design a console-based Digital Wallet application that allows users to register, log in, check balance, deposit, spend money, and view transaction history.
-
68. How would you use ArrayList or HashMap to temporarily store user data and transactions in your Digital Wallet project?
-
69. Explain how encapsulation can help secure user credentials in your Digital Wallet application.
-
70. If you had to add a feature to your Digital Wallet app that sends notifications to users when their balance is low, which Java concepts from this module would you use and how would you implement it?
- Hint:** Think about conditional statements, methods, and possibly threads for background checking.
-
71. How can you use ChatGPT or GitHub Copilot to help you debug a Java program that is giving a NullPointerException? What steps would you take to verify the suggested fix?
- Hint:** Consider the importance of understanding and testing the AI's suggestion before using it.
-
72. How could AI tools help you quickly generate boilerplate code for classes or methods in your Java project, and why is it important to review the code before using it?
-